Generative Learning

Chapter 17: **Autoencoders, GANs, and Diffusion Models**
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Discriminative vs Generative Modeling

- Discriminative models classify, generative models create.

Discriminative Modeling

- In supervised learning, the goal is to predict a class label from input features.
- These models learn to distinguish between classes.
- Learn: $P(y|x) \rightarrow$ probability of label given input.
- Focus on decision boundaries.

Generative Modeling

- Learns the underlying data distribution
- Learn: $P(x|y)$ (and sometimes $P(x)$)
- Models how data is generated, not just how to classify it
- Can generate new, realistic examples
- Strong models can produce outputs indistinguishable from real data

Deep Generative Models

- Deep generative models use deep neural networks to learn complex data distributions, typically in an *unsupervised* or *self-supervised* manner.
- Captures high-level structure and generates realistic samples.
- Examples of deep models: Restricted Boltzmann Machines (RBM), Deep Belief Networks (DBN), Variational Autoencoders (VAE), Generative Adversarial Networks (GAN).

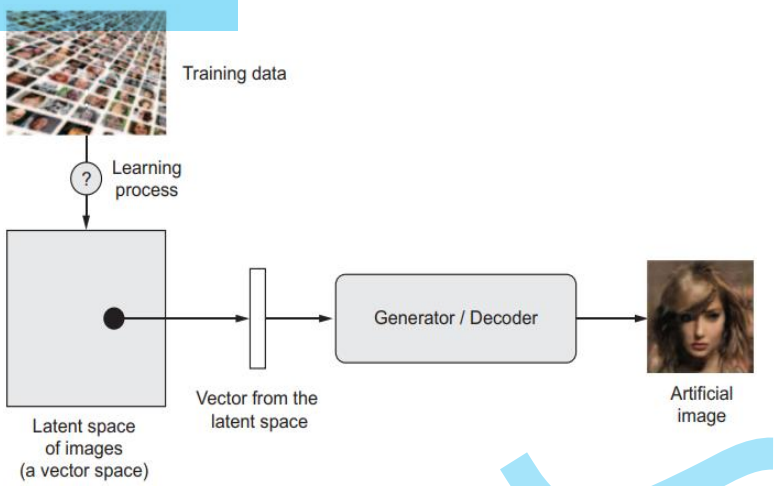
Latent Space and Sampling

- An embedding is a learned mapping of input data from high-dimensional space to a lower-dimensional vector representation.
- These vectors lie in an embedding (latent) space where distance reflects similarity between inputs.
- Generative models learn this compact representation (latent space) of the data.
- This space captures the underlying structure and patterns in the data.

- The key idea is to construct a latent space where each point corresponds to a meaningful data example.
- Once learned, we can:
  - Sample points (randomly or deliberately) from the latent space.
  - Map them back to the original data space.
  - Generate new, realistic data.

embedding space is a type of latent space

latent space : A hidden representation space learned by a model where raw input is compressed into meaningful features.



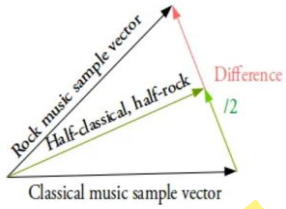
Interpolating in Latent Space

- In a continuous and structured latent space, moving along directions corresponds to meaningful changes in generated data.
- This allows interpolation and controlled attribute editing of data.

Examples

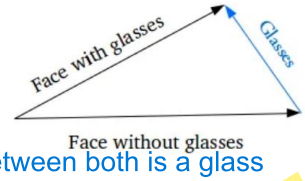
Interpolation Between Two Samples

- Given two latent vectors (e.g., rock music and classical music), we can interpolate between them:
  - Generate intermediate latent points along the line connecting them.
  - Decode these points to obtain smooth transitions between samples.



Attribute Transfer (Concept Vectors)

- Certain attributes (e.g., "glasses") can be represented as a direction in latent space.
- To learn this:
  - Take average representations of samples with and without the attribute.
  - Compute the difference vector (concept vector).
  - Add this vector to a new latent representation.
  - Decode to obtain the modified output (e.g., adding glasses to a face).



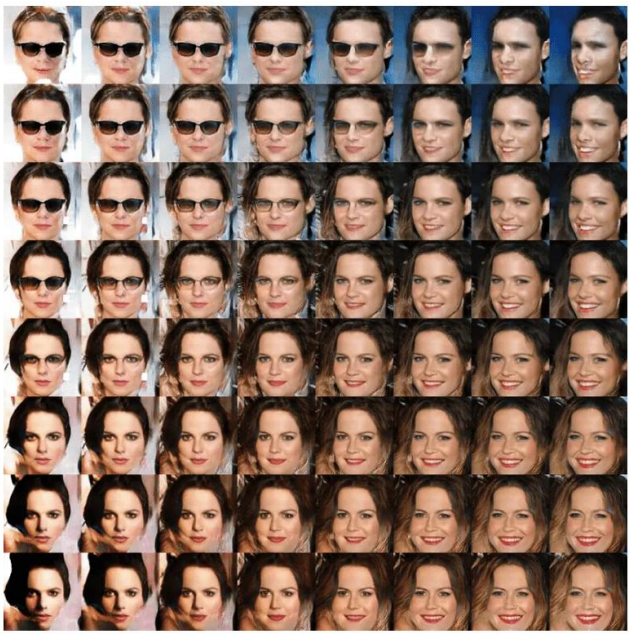
since diff between both is a glass

Facial Attributes

- Let z_1 = latent vector of a neutral face
- Let z_2 = latent vector of a smiling face
- Interpolating between z_1 and z_2 produces faces with gradually changing expressions
- Similarly, a learned vector s (smile direction) can be used:
  $z + s \rightarrow$ same face, but smiling.

Image source: Creswell, Antonia, Tom White, Vincent Dumoulin, Kai Arulkumaran, Biswa Sengupta, and Anil A. Bharath. "Generative adversarial networks: An overview." IEEE signal processing magazine 35, no. 1 (2018): 53-65.

Example of a smile vector, a concept vector discovered by Tom White from the Victoria University School of Design in New Zealand, using VAEs trained on a dataset of faces of celebrities (the CelebA dataset).

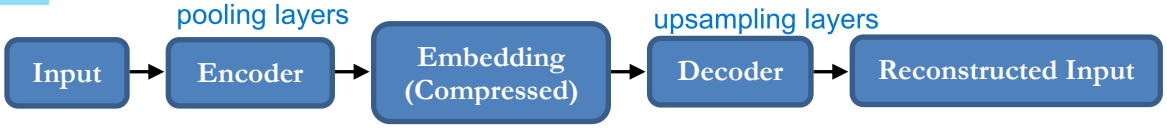


Continuous space of faces generated by Tom White using VAEs

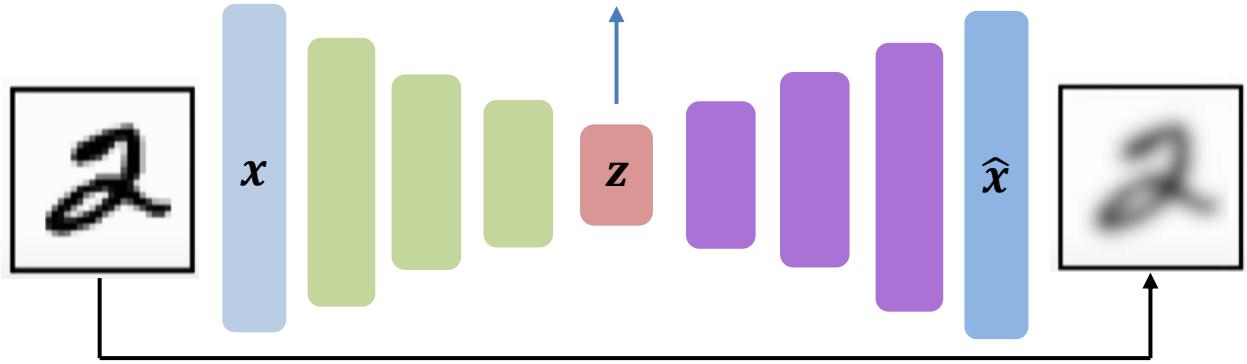
Autoencoders and Variational Autoencoders

Autoencoder

- Autoencoding means self-encoding of data.
- The concept was introduced in the paper, "Reducing the Dimensionality of Data with Neural Networks" by Geoffrey Hinton, and Salakhutdinovin Ruslan in 2006.
- An autoencoder is an *unsupervised* neural network that learns to compress input data and then reconstruct it with minimal loss.



Bottleneck hidden layer: forces network to learn a compressed latent representation.



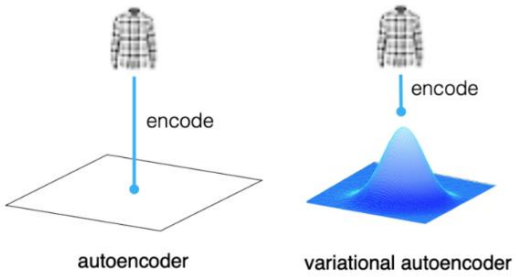
- An autoencoder has two parts:
  - Encoder
    - Maps input x to latent representation z.
    - Compresses data into a low-dimensional embedding.
  - Decoder
    - Maps latent representation z to reconstruction $\hat{x}$.
    - Tries to recover the original input.
- A CNN encoder uses convolution + pooling layers, while the corresponding decoder uses transposed convolution (deconvolution) or upsampling layers.
- Training Objective is to make reconstruction close to input.
- The loss function is : $L(x, \hat{x}) = \|x - \hat{x}\|_2^2$
  This is the reconstruction loss (often Mean Squared Error, MSE).
  Measures difference between original input x and the reconstructed output $\hat{x}$.
- This type of training does not require any labels, hence unsupervised.

- *Issue*: Autoencoders learn compression and reconstruction, but they do not guarantee a smooth or meaningful generative latent space. Random sampling from latent space often produces unrealistic or invalid outputs due to uneven distribution of encoded points (empty regions in latent space and lack of smoothness between regions). These problems worsen in higher dimensions.

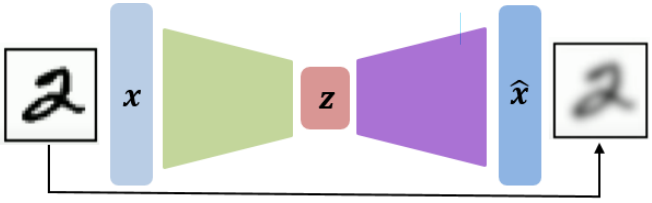
auto encoders are not generative but variational autoencoder is

Variational Autoencoder (VAE)

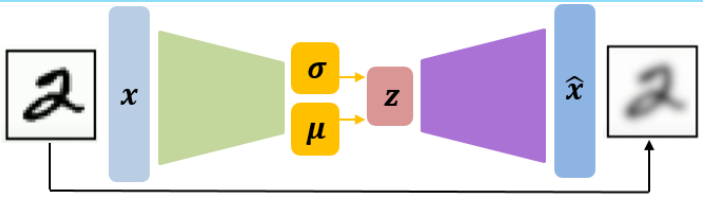
- The concept of Variational autoencoder (VAE) was introduced by Diederik Kingma and Max Welling, "Auto-Encoding Variational Bayes", arXiv preprint arXiv:1312.6114 (2013).
- VAE is generative and probabilistic model that extend the concept of autoencoders by introducing probabilistic encoding and decoding.
- It maps an input image to a distribution rather than a point in latent space.
- This means that instead of learning a single encoding for each input, the VAE learns a distribution of encodings that can represent the input image with some degree of uncertainty.
- This allows the VAE to learn a richer representation of the input data that can be used for various generative tasks.
- Hence VAEs are generative autoencoders, meaning that they can generate new instances that look like they were sampled from the training set.

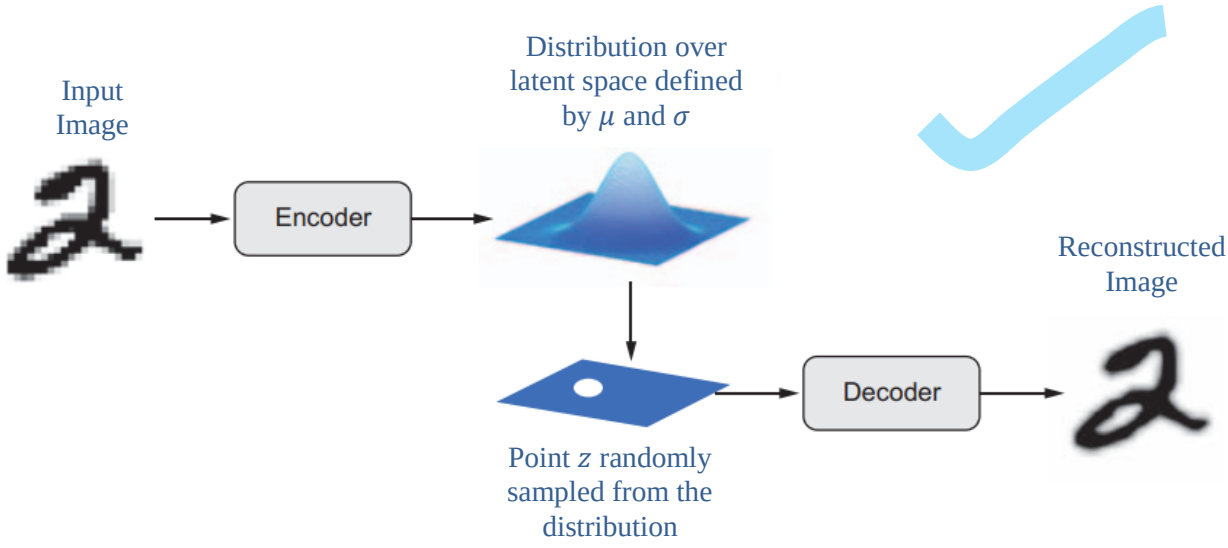


- The standard autoencoder is deterministic; weights are fixed and hence every time we pass the same input through the network we get the same deterministic output.
- In practice, their latent space is often poorly structured, not smooth, and not suitable for generation.
- For these reasons, they have largely fallen out of fashion.



- VAE introduces an element of randomness, a probabilistic twist, by adding random sampling to the bottleneck layer, which enables it to produce continuous and well-organized latent space.
- This variation allows us to generate new instances, similar to the input data but not exactly the same.





- A VAE, instead of compressing its input image into a fixed code in the latent space, turns the image into the parameters of a statistical distribution: a mean μ and a standard deviation σ. Now the encoder outputs two vectors μ and σ, instead of one vector z.
- z is then randomly sampled from μ and σ as follows:

$$z = \mu + e^{\sigma} * epsilon$$

where epsilon is a random tensor of small values.
- A decoder module maps this point in the latent space back to the original input image.
- Because epsilon is random, the process ensures that every point that's close to the latent location where you encoded x, μ can be decoded to something similar to x, thus forcing the latent space to be continuously meaningful.
  - Any two close points in the latent space will decode to highly similar images.
- Hence the encoder computes the probability $P_{\varphi}(z|x)$, and the decoder computes the probability $P_{\theta}(x|z)$, where φ and θ respectively represent the weights of the encoder and decoder.

- The loss function of VAE has two components:

$$L(\varphi, \theta, x) = Reconstruction\ loss + Regularization\ term$$

- Reconstruction Loss
  - Same as autoencoder $\|x - \hat{x}\|_2^2$.
  - Ensures output resembles input
- Regularization Term – KL Divergence
  - Regularization term, also called the VAE loss, helps learn well-formed latent spaces and reduce overfitting to the training data.
  - It asks the encoder to put the data into a known distribution in the latent space by defining a fixed prior $P(z)$ on latent distribution and training to make the inferred latent distribution $P_{\varphi}(z|x)$ as similar as possible to this prior.
  - A common choice of prior $P(z)$ is Standard Gaussian/Normal Distribution ($\mu = 0$, $\sigma^2 = 1$).
  - This distribution discourages too much divergence from the mean.
  - KL (Kullback-Leiber) divergence is given as: $-\frac{1}{2}\sum_{i=1}^{k}\left(1 + \log\left(\sigma_i^2\right) - \sigma_i^2 - \mu_i^2\right)$

    Where, k is the codings' dimensionality, and μ_i and σ_i are the mean and standard deviation of the i^{th} component of the codings.
- Hence

$$L(\varphi, \theta, x) = \|x - \hat{x}\|_2^2 - \frac{1}{2}\sum_{i=1}^{k}\left(1 + \log\left(\sigma_i^2\right) - \sigma_i^2 - \mu_i^2\right)$$

Applications of VAEs

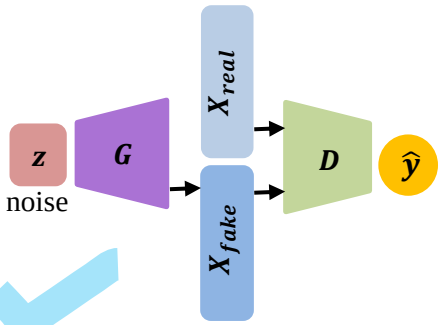
- Image Generation and Data Augmentation: Generate new images similar to training data.
- Image Denoising: VAE learns to ignore noise and keep essential structure
- Image Compression: Like a smart compression system that keeps important features
- Interpolation / Morphing: Smoothly transform one image into another (Example: neutral face to smiling face).

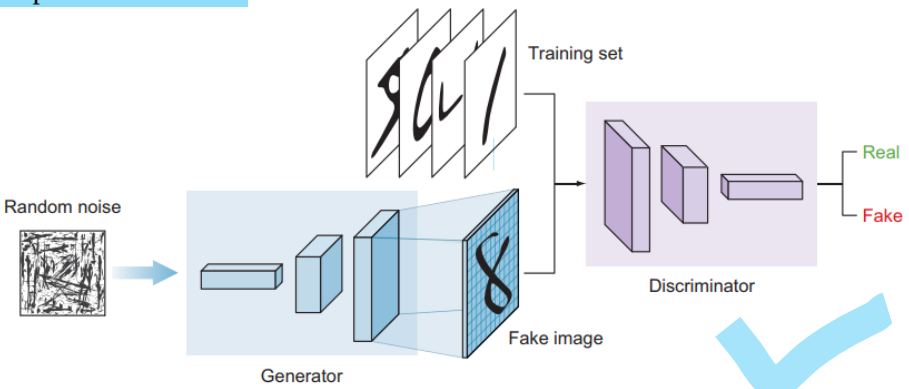
Generative Adversarial Networks

- Generative adversarial networks (GANs) were introduced by Ian J. Goodfellow et al., in the paper titled "Generative Adversarial Networks" in 2014.
- A key strength of GANs is their ability to generate highly realistic images, videos, music, and text.

GAN Architecture

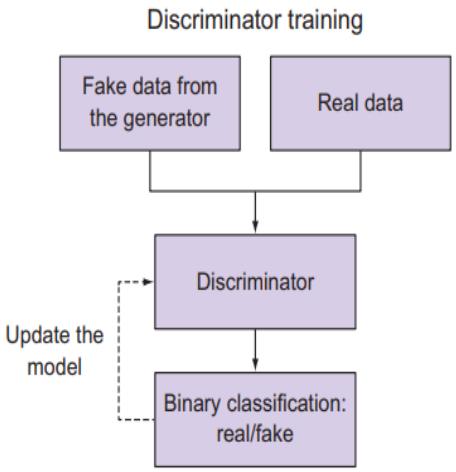
- GANs are based on adversarial training and consist of two neural networks:
  - Generator (G): Takes random noise as input and generates data resembling the training distribution, attempting to fool the discriminator.
  - Discriminator (D): Receives both real and generated data and predicts whether each input is real (1) or fake (0).
- The generator and discriminator compete in a zero-sum game:
  - If the discriminator correctly distinguishes real and fake data, it improves while the generator is penalized.
  - If the generator successfully fools the discriminator, it improves while the discriminator is penalized.
- The generator learns a mapping from noise to the data distribution, while the discriminator acts as a classifier.
- Typically, the discriminator is a CNN that reduces dimensionality, and the generator is an inverted CNN that upsamples from a latent vector to the image space.
- After training, only the generator is used to produce new data.



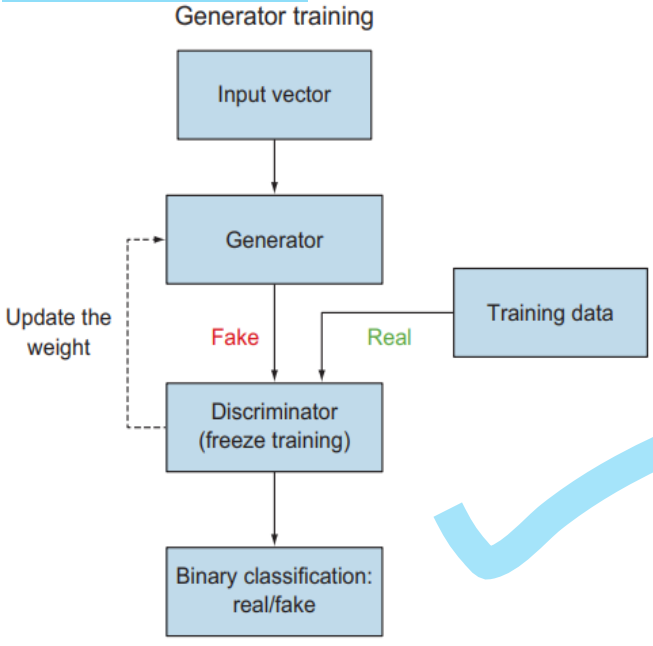


Training a GAN

1. Train the Discriminator
   - Train on labeled real and fake data.
   - The objective is to correctly classify real vs fake.
   - Only discriminator weights are updated.



2. Train the Generator
   - Use a combined model (generator + frozen discriminator).
   - Generator produces fake images from noise.
   - Discriminator evaluates them.
   - Generator updates its weights based on how well it fools the discriminator.



- Initially, the discriminator performs well because generated samples are poor.
- Over time the discriminator improves at classification, and the generator improves at producing realistic samples.
- Training continues until generated data becomes difficult to distinguish from real data.

Optimization Objective

- GANs optimize a minimax objective:

$$\min_G \max_D V(G,D) = E_{x\sim P_{data}(x)}[\log(D(x))] + E_{z\sim P_z(z)}[\log(1 - D(G(z)))]$$

Where

G	Generator Model
D	Discriminator Model
z	Random Noise (Fixed size input vector)
x	Real Image
$D(x))$	Discriminator's output when the real image is an input
$G(z)$	Image generated by Generator (Fake Image)
$D(G(z)))$	Discriminator's output when the generated image is an input
$P_{data}(x)$	Probability Distribution of Real Images
$P_z(z)$	Probability Distribution of Fake Images
$E_{x\sim P_{data}(x)}[\log(D(x))]$	Average log probability of D when real image is input.
$E_{z\sim P_z(z)}[\log(1 - D(G(z)))]$	Average log probability of D when the generated image is input.

- Discriminator's goal:
  - Maximize $D(x)$ (real → 1)
  - Minimize $D(G(z))$ (fake → 0)
  - Effectively maximize both first and second terms
- Generator's goal:
  - Maximize $D(G(z))$ (fool the discriminator)
  - Effectively minimize the second term

Evaluating GANs

- Evaluating GANs is challenging because there is no clear convergence criterion or universally accepted metric.
- This makes it difficult to:
  - Decide when to stop training
  - Compare models
  - Tune hyperparameters
- Two common evaluation methods are:
  1. Manual Inspection
  - Generated samples are visually inspected.
  - Example: Tim Salimans et al. (2016) used human annotators via Amazon Mechanical Turk to distinguish real from fake images.
  - However, results can vary depending on annotator skill, feedback, and task setup.
  2. Inception Score
  - Measures two properties using a pre-trained classifier:
    - Image quality: Clear, classifiable outputs
    - Diversity: Variety across different classes
  - A high score indicates that each image is meaningful and the overall set is diverse.

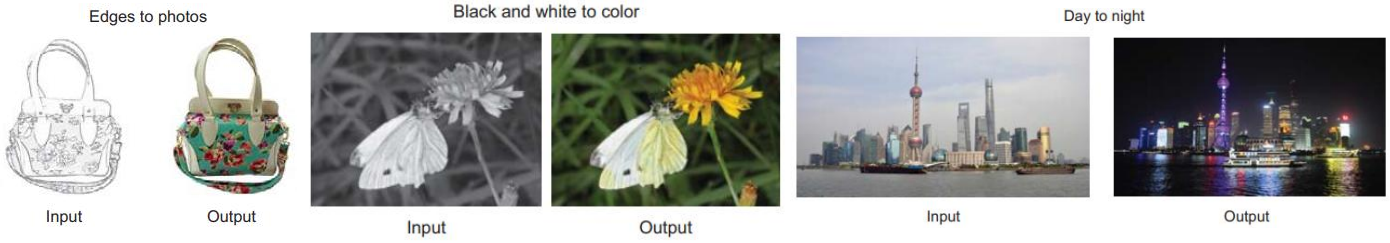
Applications of GANs

Image Super-Resolution (SRGAN)

- *Super-Resolution GAN (SRGAN)* converts low-resolution images into high-resolution, photo-realistic outputs.
- Proposed by Christian Ledig et al. (2016) in "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network."

Image-to-Image Translation

- Transforms one representation of a scene into another using paired training data.
- Common applications include: day ↔ night conversion, summer ↔ winter translation, sketches → realistic images, and grayscale → color images.
- One implementation is *Pix2Pix* (Conditional GAN). introduced by Phillip Isola et al. (2016) in "Image-to-Image Translation with Conditional Adversarial Networks."



Text-to-Image Synthesis

- Generates images from textual descriptions, a challenging task requiring both semantic understanding and visual realism.
- One implementation is *StackGAN*, proposed by Zhang et al. (2016) in "StackGAN: Text to Photo-Realistic Image Synthesis with Stacked Generative Adversarial Networks."